

#07 Brute Force & Recursion

김성수

4thrunyul@gmail.com

0. 목차



1. Ice breaking

- 교육부의 어려운 문제

2. Brute Force Algorithm

3. Function

4. Recursion

1. Ice breaking

1. Ice breaking: 교육부의 어려운 문제



문제

하워드와 필립, 조이스 세 사람 중에 한 사람은 착한 사람, 한 사람은 나쁜 사람, 나머지 한 사람은 사기꾼이다.

착한 사람은 진실만을 말하고 나쁜 사람은 거짓말만 한다.
사기꾼은... 진실과 거짓 어느 쪽도 서슴지 않는다.

어느날, 조이스는 “필립은 착한 사람이 아니면 나쁜 사람이다.”라고 말했다.
그러자 필립은 “하워드와 조이스 중 착한 사람이 한 명 있다.”라고 말한다.

세 사람 중 누가 사기꾼일까?

1. 하워드
2. 필립
3. 조이스

1. Ice breaking: 교육부의 어려운 문제



전제

1. 착한 사람, 나쁜 사람, 사기꾼이 한 명 씩 있음.
2. 조이스 : 필립은 착한 사람이 아니면 나쁜 사람이다.
3. 필립 : 하워드와 조이스 중 착한 사람이 한 명 있다.

풀이

필립이 착한 사람이라면? : 조이스와 필립이 모두 착한 사람이라 전제1을 배반함.

필립이 나쁜 사람이라면? : 하워드와 조이스 중에 착한 사람이 없다는 말이 참이 되어 셋 중 착한 사람이 없어지고 전제1을 배반함.

정답

- 사기꾼 : 필립
- 착한 사람 : 하워드
- 나쁜 사람 : 조이스

1. Ice breaking: 교육부의 어려운 문제

문제

설원의 어느 눈에 띄지 않는 구석에 고대에서 전해져 내려오는 보물이 숨겨져 있다고 한다.
탐험가 파란 머리 멧쟁이 삼포가 천신만고 끝에 보물이 숨겨진 신비한 장소를 찾게 된다.

그곳에는 말할 수 있는 문이 2개 있는데, 하나는 금, 하나는 은으로 되어 있다.
2개의 문 중 하나는 진실을 말하고 나머지 한 개는 거짓을 말한다.
2개의 문 중 한 문 뒤에는 보물이 숨겨져 있고 나머지 한 문 뒤에는 고대 괴물이 있다.
보물 지도에 적힌 기록에 의하면, 2개의 문은 한가지 질문에만 대답한다고 한다.

영리한 파란 머리 멧쟁이 삼포는 잠시 생각하다가 은색 문에 “금색 문은 은색 문 뒤에 무엇이 있다고 말할까?”라고 질문한다.
은색 문은 “금색 문은 은색 문 뒤에 보물이 숨겨져 있다고 할 거야”라고 대답한다.
파란 머리의 멧쟁이 삼포는 어떤 문을 선택해야 할까?

1. 금색 문
2. 은색 문

1. Ice breaking: 교육부의 어려운 문제



풀이

- 은색 문이 진실을 말한 다면? : “금색 문은 은색 문 뒤에 보물이 있을 거야”가 참이기 때문에 금색 문이 거짓말을 하는 문
 - 거짓말을 하는 문이 “보물”이라고 말하는 것은 실제로는 “괴물”
 - 따라서 은색 문 뒤 : 괴물, 금색 문 뒤 : 보물
- 은색 문이 거짓을 말한 다면? : “금색 문은 은색 문 뒤에 보물이 있을 거야”가 거짓이기 때문에 금색 문이 진실을 말하는 문
 - 진실을 말하는 문이 실제로 의미하는 바는 “은색 문 뒤에는 괴물”
 - 따라서 은색 문 뒤 : 괴물, 금색 문 뒤 : 보물

정답

- 금색 문

2. Brute Force Algorithm

자전거 자물쇠 비밀번호를 까먹어버렸다?

- 이때 어떻게 하면 풀 수 있을까?
- 0000부터 9999까지 전부 맞춰보기



2. Brute Force Algorithm



Brute Force Algorithm(브루트 포스 알고리즘)

- 정의 가능한 모든 경우의 수를 모두 확인하여 정답을 찾는 방식으로 ‘완전 탐색’이라는 명칭으로도 불림
- 완전성 : 이론적으로 가능한 모든 경우의 수를 탐색하므로, 해가 존재한다면 반드시 찾을 수 있음.
- 단순성 : 복잡한 최적화 논리나 수학적 기법 없이 문제의 조건 그대로 구현이 가능함.
- 비효율성 : 입력값의 크기가 커질수록 확인해야 할 경우의 수가 기하급수적으로 늘어남.

2. Brute Force Algorithm



Brute Force Algorithm(브루트 포스 알고리즘)

- 문제를 풀기 전 전체 경우의 수를 계산해 시간 제한 내 연산이 가능한지 확인
- C++에서는 일반적으로 1초에 약 1억 번 연산 가능
 - $O(n^2) : n \leq 10,000$ - 이중 반복문
 - $O(2^n) : n \leq 20$ - 부분집합 탐색
 - $O(n!) : n \leq 10$ - 순열 탐색

2. Brute Force Algorithm



실습(BOJ)

- [2798 블랙잭](#)
- [2231 분해합](#)
- [2501 약수 구하기](#)

2. Brute Force Algorithm



실습(정올)

- [1504 Blackjack](#)
- [2605 덩치](#)
- [8642 쉬운 최장 증가 부분 수열](#)
- [1201 제곱수의 합](#)

2. Brute Force Algorithm



1210 제곱수의 합

- 배경 지식 : 라그랑주 네 제곱수 정리
- 모든 자연수 N 은 최대 4개의 제곱수 합으로 나타낼 수 있다.
 - $N = a^2 + b^2 + c^2 + d^2$ ($a, b, c, d \in \mathbb{Z}, 0 \leq Z$)
- 즉, 답의 상한이 4로 고정 됨.
 - 1, 2, 3개 항으로 N 을 만들 수 있는지 순서대로 검사
 - 끝까지 만들어지지 않는다면 남은 가능성인 4를 정답으로 확정
- 이때, $N = 100,000$ 이므로 단일 제곱수의 최대값은 $\sqrt{100,000} \approx 316$ 이다.
- $316 * 316 * 316 = 31,554,496$ 이므로 1억 번을 넘지 않으므로 브루트 포스로 해결할 수 있음.

3. Function

3. Function



함수(Function)

- 특정 작업을 수행하기 위해 설계된 독립적인 코드 블록.
- 프로그램의 모듈화를 가능하게 하며 코드의 재사용성과 가독성을 높이는 핵심 요소.

3. Function

```
1  #include <iostream>
2
3  using namespace std;
4
5  int calc(int x, int y) {
6      return x + y;
7  }
8
9  int main() {
10     int a, b, result;
11
12     cin >> a >> b;
13
14     result = calc(a, b);
15
16     cout << result;
17
18     return 0;
19 }
```

반환값이 있는 함수

- 작업 결과를 호출자에게 돌려주는 함수
- return을 통해 반환해 줄 데이터를 명시해야 함.

int : 반환 타입

calc : 함수 명

(int x, int y) : 매개변수(파라미터, 입력값)

result : 반환할 값

```
Input:                                     Copy
1 2
Expected Output:                           Copy
3
```


3. Function

```
1  #include <iostream>
2
3  using namespace std;
4
5  void state(int x, int y) {
6      cout << "현재 레벨: " << x << "\n";
7      cout << "현재 체력: " << y;
8  }
9
10 int main() {
11     int a = 10;
12     int b = 90;
13
14     cin >> a >> b;
15
16     state(a, b);
17
18     return 0;
19 }
```

반환값이 없는 함수

- 특정 로직만 수행하고 값을 돌려주지 않는 함수

Input:

Copy

Expected Output:

Copy

현재 레벨: 10

현재 체력: 90

3. Function



실습(BOJ)

- [15964 이상한 기호](#)
- [28444 HI-ARC=?](#)

3. Function



실습(정올)

- [576 함수1 - 자가진단6](#)
- [177 함수2 - 형성평가3](#)

4. Recursion

4. Recursion



Recursion(재귀)

- 문제를 동일한 형태의 하위 문제로 줄여 나가는 축소(Reduction) 기법
- 사고 방식
 - 작게 축소된 문제들은 프로그램이 ‘알아서 잘’ 처리해줄 것이라 믿고 코드를 작성하는 것이 중요한 사고 방식임. 수학적으로는 ‘귀납적 가정’이라고 함.
 - 재귀 함수를 잘 사용하기 위해서는 기존의 ‘절차 지향적 사고’를 탈피해야 할 필요가 있음.
- 우리의 관심 사항
 - 원래 문제를 어떻게 더 작은 단위로 단순화 할지
 - 가장 작은 단위가 되었을 때 어떤 값을 반환하거나 출력 할지

4. Recursion

```
1  #include <iostream>
2
3  using namespace std;
4
5  long long factorial(int n) {
6      if (n <= 1) return 1;
7
8      return n * factorial(n - 1);
9  }
10
11 int main() {
12     int n = 10;
13
14     long long result = factorial(n);
15
16     cout << result;
17
18     return 0;
19 }
```

절차 지향적 사고

- f(10) 호출, $10 * f(9)$
- f(9) 호출, $9 * f(8)$
- f(8) 호출 ...

귀납적 사고

- f(n) 호출, $n * f(n - 1)$

4. Recursion

Recursion Function(재귀 함수)

- 함수 내부에서 자기 자신을 호출하는 함수
- 이론적으로 모든 재귀 함수는 반복문으로 표현할 수 있음
 - 역으로 모든 반복문을 재귀 함수로 표현할 수 있음.
 - 재귀는 반복문으로 구현했을 때에 비해 코드가 간결하지만 메모리/시간에서는 손해

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n = 10;
7
8      long long result = 1;
9
10     for (int i = 1; i <= n; i++) {
11         result *= i;
12     }
13
14     cout << result;
15
16     return 0;
17 }
```

```
1  #include <iostream>
2
3  using namespace std;
4
5  long long factorial(int n) {
6      if (n <= 1) return 1;
7
8      return n * factorial(n - 1);
9  }
10
11 int main() {
12     int n = 10;
13
14     long long result = factorial(n);
15
16     cout << result;
17
18     return 0;
19 }
```

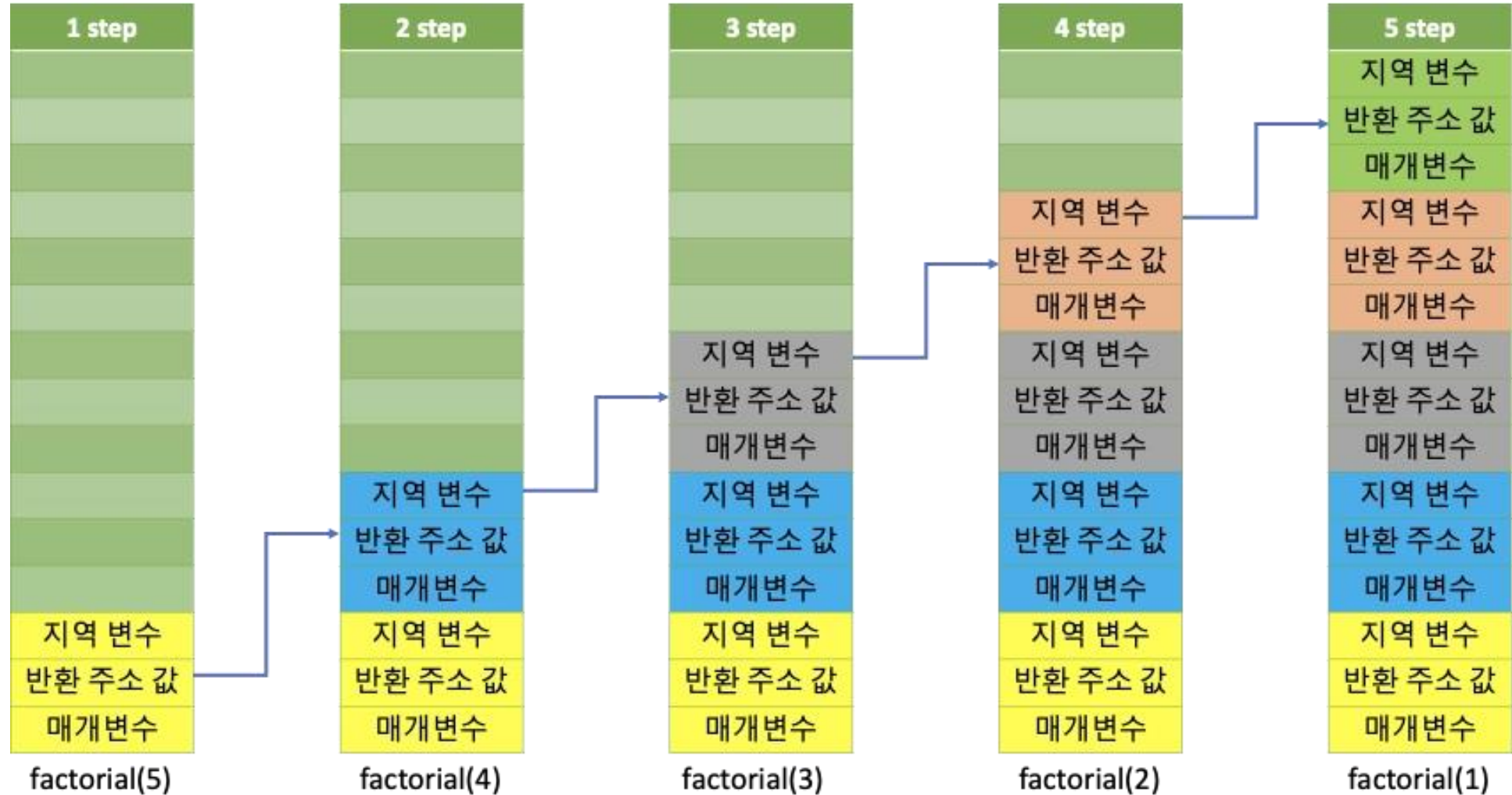
4. Recursion



Recursion Function(재귀 함수)

- 재귀 함수가 자기 자신을 부를 때 스택 영역에 계속 누적이 됨
- 만약 이 재귀 함수가 스택 영역을 벗어나게 된다면 Runtime Error 발생
- 스택 영역은 로컬 환경 마다 다르고 직접 설정할 수 있어서 내 컴퓨터에서 오류가 발생하는 게 다른 컴퓨터나 OI에서는 잘 돌아갈 수 있음

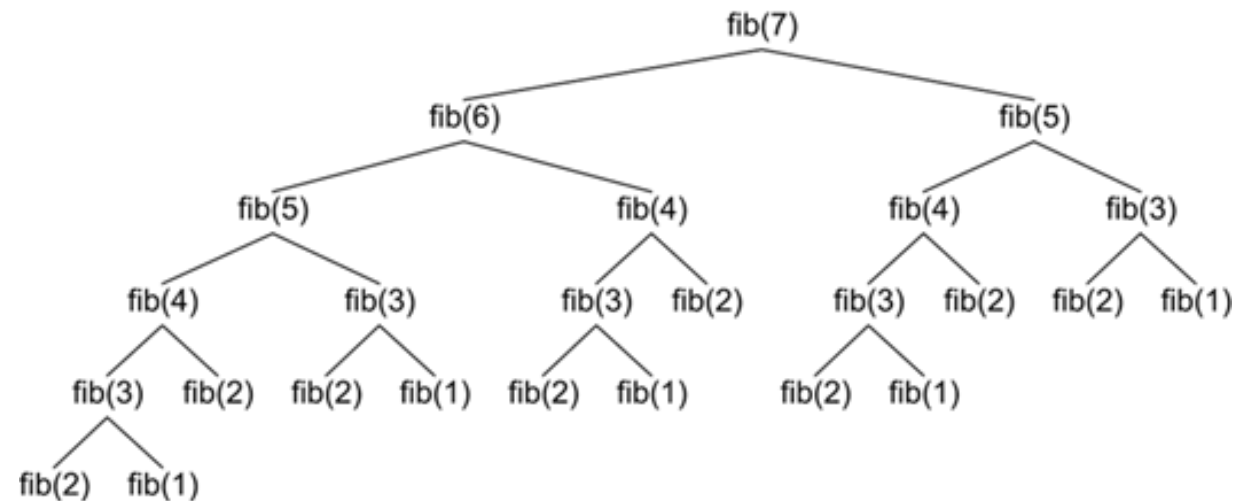
4. Recursion



4. Recursion

Recursion Function(재귀 함수)

- 기저 조건(Base condition)
 - 특정 입력에 대해서는 자기 자신을 호출하지 않고 종료되어야 함.
 - 모든 입력은 기저 조건으로 수렴해야 함.
- 함수의 인자로 어떤 것을 받고 어디까지 계산하고 자기 자신을 호출할지 정확하게 정의해야 함.
- 자기 자신을 여러 번 호출하게 되면 비효율적일 수 있음



4. Recursion



실습(기초)

- [9101 함수3 - 연습문제1](#)
- [231 함수3 - 형성평가1](#)
- [1309 팩토리얼](#)

4. Recursion



실습(응용)

- [1169 주사위 던지기1](#)
 - [코드](#)
- [1175 주사위 던지기2](#)
 - [코드](#)
- [2567 사이클](#)
- [1021 장난감조립 \(easy\)](#)

4. Recursion



실습(심화)

- [1161 하노이1](#)
 - [하노이 탑 해보기](#) : 원판이 3, 4, 5인 경우 모두 깨고 문제 풀어보기
 - [코드](#)
- [1459 숫자고르기](#)